

What is left, and what is needed

*Delta Rentals AI sales agent — status against their scope,
24 August 2026*

23 August 2026

Status against [ai sales agent scope for developers.pdf](#), as of 24 August 2026.

The agent runs on Delta's real fleet, answers from Delta's published terms, sells in the patterns taken from their own conversations, holds a car rather than promising one, escalates what it cannot answer, and takes a payment. **Sections 2, 3 and 4 of their scope are complete.** 791 tests, 70 commits.

What remains is three decisions, two pieces of code, and four things only they can hand over.

Where each requirement stands

Section 1 — Conversational AI Layer Knowledge-base driven and never inventing a figure: done, and it is the strongest part of the build. Sales script and objection handling: done, written from their thirteen conversations. **Not done: it runs on Gemini's free tier rather than Claude or GPT, and the free tier measured 54 seconds against their 2-5 second target.** One decision fixes both.

Section 2 — WhatsApp-native behaviour — complete. Direct Cloud API, typing indicators, reactions, photo sets, natural pacing, and payment links verified end to end: a real Stripe page at AED 1,887.90, paid, and the booking marked paid by the callback.

Section 3 — Escalation and human in the loop — complete, and beyond what they asked. They assumed two kinds of case; there are three. A decision the owner picks, a handover where a person takes over, and an **answer**, where the owner supplies a figure nobody else has and the agent relays it exactly.

Section 4 — Improvement over time — complete, and it was explicitly not a day-one requirement. Every conversation is tagged booked, dropped or escalated, and a benchmark replays their own thirteen conversations against the agent so a prompt change can be measured rather than guessed at.

Section 5 — Data source of truth — the one section that cannot be closed from this side. Their scope rules out working from a public website, which is exactly where the fleet currently comes from, and availability is published nowhere at all. Holds are the honest workaround, not a solution.

Part one — decisions only you can make

1. Who pays the running cost. Roughly AED 400-550 a month for the model, plus hosting. Nothing else in the project is blocked on money, and this one blocks two requirements at once: moving off the free tier is what makes it Claude and what makes it hit 2-5 seconds. Their scope leaves the engagement structure "to be discussed" — this belongs in that conversation rather than being absorbed quietly.

2. Where it runs, and in whose account. Railway, Render or Fly each provide a database on the same network. The account should be theirs: it will hold their customers' names, numbers and complete conversation histories.

3. Whether to answer their four questions now. Their scope asks four questions directly and expects them answered in the reply. Three can now be answered with a working system rather than a claim — see part four.

Part two — code left on your side

Live now, nothing blocking:

- **Quoting accuracy.** The benchmark found the agent quoting three days for a two-day rental, and correcting itself only when asked to show the calculation. It also said a car was available and then, two messages later, that it was booked out. Roughly half a day, and the benchmark measures whether a fix works.
- **Host the vehicle photographs.** Meta fetches images over HTTPS, so `assets/vehicles/` has to be public before a real number works. About an hour, and it needs a GitHub account.

Waiting on their answers, deliberately:

- **The de-demo pass** — 243 references across 23 files, the `DEMO-` prefixes and the demonstration footer. Half a day, and it must happen *after* their figures are confirmed, because the disclosure and the real data have to flip together.
- **A payments table.** Payment is currently a field on a booking, which fits one payment per booking. Their own conversations show a holding payment and then a balance, which does not fit. Cheap to change now, awkward once there are real payments in the database. Question 22 decides it.

Part three — what is needed from Delta

Three of these are things their own scope says they would provide.

1. WhatsApp Business account access — promised in their scope, not yet sent. Without it there is no live product. Never register a personal number with the Cloud API; it removes that number from WhatsApp permanently.

2. One approved message template, named `case_update`. Only they can request it, and Meta takes a day or two, which is why it should be asked for first. Without it, an owner answering an escalated case the next morning cannot reach that customer at all.

3. A maintained source for pricing and availability — their section 5. Their team checks availability by hand; whatever they check, we need to read. This is the single largest unknown in the project: a spreadsheet is an afternoon, a fleet management system with an API is an integration nobody can size unseen.

4. The figures their website does not publish — `policy-questionnaire.pdf`, now written as confirmation rather than blanks, because their own conversations answer most of it. Their team quotes AED 500 + VAT for the no-deposit option, AED 5,000 deposit held 28 working days, an excess "from AED 3,500" where their terms say 5,000, and AED 15 per extra kilometre where their terms say the range starts at 25.

5. Which of their two rental agreements is current. Their terms page carries two, disagreeing on about twelve figures. One reply resolves all of them. It is question 26, and it is the highest-value question on the list.

6. Their sales playbook and objection script — promised, not sent. The playbook in use was written from their conversations instead, which is a decent substitute and not the same thing.

7. Their Stripe account, or the answer that they do not want payment links. Their scope asks for links; their terms say payment happens at collection and in thirteen conversations nobody ever sent one. Question 22.

8. Sign-off on the escalation rules. Their scope asks for configurable rules for what escalates. Configurable is built; deciding what the agent may settle alone is a business call nobody has made. Every discount ceiling is currently zero, which is safe and will exhaust them within a week.

9. Who answers escalations — a number, and the hours a person is reachable.

Part four — the four questions their scope asks you

Unanswered so far, and the only part of their document with nothing produced against it.

"Have you built directly on Meta's Cloud API before, not just through a no-code tool?" — the answer is this system: raw webhook, signature verified on the raw body, own send client, typing indicators, reactions, interactive buttons, the 24-hour window and template re-opening.

"How would you architect the escalate-to-owner flow?" — three kinds of case, a decision recorded against the customer it belongs to, the owner's answer relayed verbatim, timeouts that tell the customer honestly, and the 24-hour window handled.

"What is your honest take on the learns-over-time requirement?" — they wrote this one as a filter. The answer is a gated, replay-verified, human-reviewable loop, plus a benchmark that replays their own conversations so a change is measured instead of asserted. Nothing about it is autonomous, which is the point.

"Timeline and cost for phase 1 versus phase 2?" — the only one of the four that needs a decision from you rather than a description. Both phases are substantially built.

The order that actually unblocks things

1. Send the questionnaire, and ask for the number and the template. Meta's approval is the longest lead time in the project. **2. Answer their four questions** while waiting. **3. Decide the running cost**, then switch the provider and measure the latency against their target. **4. Fix quoting accuracy** and host the photographs — neither needs anyone else. **5. When their figures arrive:** encode them, set the discount ladder, de-demo, deploy.

Everything on that list past step one starts with something sent today.